

URL DOWNLOADER

Complete Manual API Testing,
Learning & Validation Framework

Master Codex Prompt

A structured guide for project analysis, endpoint discovery, manual testing, Postman, automation, CI/CD, and professional reporting.

Prepared as a print-ready reference document

You are acting as a Senior QA Engineer, API Testing Mentor, Manual Tester, API Automation Engineer, and Test Architect.

I have already built an application called:

URL Downloader

My goal is NOT simply to automate the APIs.

My goal is to use my own application to learn complete API testing from a manual tester's perspective, understand every endpoint, understand why each test is required, execute the tests myself, identify defects, document them professionally, and then gradually automate the same tests.

1. PRIMARY OBJECTIVE

Analyze the entire existing URL Downloader application.

Do NOT assume how the application works.

First understand the actual codebase, architecture, frontend, backend, routes, controllers, services, models, database interactions, authentication, validation, error handling, external integrations, and configuration.

Then build a complete API testing system around the existing application.

The learning progression must be:

```
graph TD; A[Application Understanding] --> B[API Discovery]; B --> C[Manual API Testing]; C --> D[Positive Testing]; D --> E[Negative Testing]; E --> F[Boundary Testing]; F --> G[Validation Testing]; G --> H[Authentication & Authorization Testing]; H --> I[Security Testing]; I --> J[Error Handling Testing]; J --> K[Response Validation]; K --> L[Database / Data Validation]; L --> M[Integration Testing]; M --> N[Performance / Basic Load Testing]; N --> O[Postman Collection]; O --> P[API Test Automation]; P --> Q[CI/CD Execution]; Q --> R[Professional API Test Report];
```

Application Understanding
↓
API Discovery
↓
Manual API Testing
↓
Positive Testing
↓
Negative Testing
↓
Boundary Testing
↓
Validation Testing
↓
Authentication & Authorization Testing
↓
Security Testing
↓
Error Handling Testing
↓
Response Validation
↓
Database / Data Validation
↓
Integration Testing
↓
Performance / Basic Load Testing
↓
Postman Collection
↓
API Test Automation
↓
CI/CD Execution
↓
Professional API Test Report

The most important requirement:

Teach me what is being tested and why, instead of only generating code.

2. FIRST STEP — ANALYZE THE PROJECT

Before changing anything, inspect the complete project.

Identify:

- Frontend framework
- Backend framework
- Programming language
- API framework
- Database
- Authentication mechanism
- Authorization mechanism
- Environment configuration
- API base URL
- Controllers/routes
- Services
- Models/entities
- Middleware
- Validation
- Error handling
- External APIs
- File handling
- URL processing/downloading logic
- Background jobs
- Queues if any
- Caching if any
- Logging
- Configuration files
- Docker configuration if present
- Existing tests
- Existing API documentation
- Swagger/OpenAPI if present
- Postman collections if present

Do not modify application functionality during this analysis.

Create:

3. API ENDPOINT DISCOVERY

Automatically discover EVERY API endpoint from the source code.

Search for:

- GET
- POST
- PUT
- PATCH
- DELETE
- HEAD
- OPTIONS

Also identify:

- Route parameters
- Query parameters
- Request headers
- Request body
- Form-data
- Multipart requests
- Authentication headers
- Cookies
- JWT
- API keys
- Request dependencies
- Response structure
- HTTP status codes
- Error responses

Create:

Use a table like:

ID	Method	Endpoint	Purpose	Auth	Parameters	Request Body	Expected Success	Expected Errors
-----	-----	-----	-----	-----	-----	-----	-----	-----
API-001	POST	/api/...	...	Yes/No	...	...	200/201	400/401/422

Do not invent endpoints.

Every endpoint must be backed by the actual source code.

4. CREATE AN API MAP

Create:

`API_TESTING/02_API_MAP.md`

Explain the relationship between endpoints.

For example:

```
Create Download Request
  ↓
Validate URL
  ↓
Create Download Job
  ↓
Process Download
  ↓
Check Download Status
  ↓
Retrieve Download
  ↓
Delete Download
```

Adapt this diagram to the actual application.

Also identify:

- independent APIs
- dependent APIs
- chained APIs
- APIs requiring previous API responses
- APIs modifying shared data

5. MANUAL API TESTING TRAINING

This is extremely important.

For EACH endpoint, create a manual testing guide.

Example:

```
Endpoint:
POST /api/download

Purpose:
Creates a download request.

Method:
POST

Headers:
Content-Type: application/json

Request:
{
  "url": "https://example.com/file.pdf"
```

```
}
```

Then explain:

What am I testing?

Why am I testing it?

What should I send?

What should I expect?

What could go wrong?

How do I identify a bug?

What evidence should I capture?

What should I report to the developer?

Do this for EVERY endpoint.

Create:

[API_TESTING/03_MANUAL_API_TESTING_GUIDE.md](#)

6. POSITIVE TESTING

For every endpoint identify valid scenarios.

Examples:

- valid URL
- valid request body
- valid authentication
- valid content type
- valid parameters
- valid resource ID
- valid file
- valid response

For each test case include:

```
Test Case ID
Endpoint
Method
Objective
Precondition
Headers
Request
Steps
Expected Result
Actual Result
```

```
Status Code
Response Validation
Pass/Fail
Defect ID
```

Create:

[API_TESTING/test-cases/positive-tests.md](#)

7. NEGATIVE TESTING

For EVERY endpoint generate negative scenarios.

Do not just test one invalid request.

Test:

- missing required field
- null value
- empty string
- whitespace
- incorrect data type
- invalid URL
- malformed URL
- unsupported URL
- invalid ID
- nonexistent ID
- duplicate request
- invalid JSON
- malformed JSON
- missing header
- incorrect Content-Type
- invalid authentication
- expired authentication
- insufficient permissions
- invalid query parameter
- invalid path parameter
- unsupported HTTP method
- oversized input
- unexpected fields

Create:

[API_TESTING/test-cases/negative-tests.md](#)

8. BOUNDARY VALUE TESTING

For every input field determine:

- minimum value
- maximum value
- minimum - 1
- minimum
- minimum + 1
- normal value
- maximum - 1
- maximum
- maximum + 1

For strings test:

- empty
- 1 character
- normal length
- maximum length
- maximum + 1
- very large input

For numbers:

- 0
- negative
- minimum
- maximum
- decimal
- very large number

For URLs specifically consider:

- HTTP
- HTTPS
- localhost
- IP address
- URL with query parameters
- URL with fragments
- URL with special characters
- URL with encoded characters
- URL with Unicode
- very long URL

- invalid scheme
- unsupported protocol
- URL without protocol
- malformed URL

Create:

`API_TESTING/test-cases/boundary-tests.md`

Do not assume limits.

First inspect the application validation rules and document the actual limits.

9. AUTHENTICATION TESTING

If authentication exists, test:

1. Valid token
2. Missing token
3. Invalid token
4. Expired token
5. Modified token
6. Empty token
7. Wrong authentication scheme
8. Token belonging to another user
9. Access without authentication

For every scenario explain:

Request
Expected Status
Expected Response
Security Risk
Why This Test Matters

Create:

`API_TESTING/04_AUTHENTICATION_TESTING.md`

10. AUTHORIZATION TESTING

Check whether users can access resources that do not belong to them.

Test:

- User A accessing User B data
- User without permission accessing protected API
- Admin-only API accessed by normal user
- Resource ID manipulation

- Horizontal privilege escalation
- Vertical privilege escalation

Do NOT perform destructive attacks against external systems.

Only test the local application and its authorized test data.

Create:

[API_TESTING/05_AUTHORIZATION_TESTING.md](#)

11. HTTP STATUS CODE VALIDATION

For every endpoint determine the expected status codes.

Validate appropriate cases such as:

```
200 OK
201 Created
202 Accepted
204 No Content
400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
405 Method Not Allowed
409 Conflict
415 Unsupported Media Type
422 Unprocessable Entity
429 Too Many Requests
500 Internal Server Error
502 Bad Gateway
503 Service Unavailable
```

Do not force every endpoint to return every status code.

Only test status codes that logically apply to the actual implementation.

Document:

[API_TESTING/06_STATUS_CODE_MATRIX.md](#)

12. RESPONSE VALIDATION

For every endpoint validate:

Status code

Response headers

Content-Type

Response body

JSON structure

Required fields

Data types

Null handling

Error structure

IDs

Timestamps

URLs

Download information

Pagination if applicable

Metadata

Response consistency

Create expected response examples based on the actual API.

13. JSON SCHEMA VALIDATION

For every JSON response identify:

```
Required fields
Optional fields
Field types
Nested objects
Arrays
Nullable fields
Allowed values
```

Create schemas where useful.

Explain schema validation in simple manual-testing language first.

Then show how the same validation can later be automated.

14. REQUEST VALIDATION

For every request test:

```
Required fields
Optional fields
Data types
Allowed values
Invalid values
Missing fields
Extra fields
Null values
Empty values
```

Boundary values
Special characters
Unicode
Large input

Document actual behavior from the application.

15. URL DOWNLOADER SPECIFIC TESTING

Because this application is a URL Downloader, specifically inspect and test:

URL validation

- valid URL
- invalid URL
- HTTP URL
- HTTPS URL
- unreachable URL
- timeout URL
- redirect URL
- URL with authentication
- URL with query parameters
- URL with fragments
- URL with special characters
- URL with encoded characters
- very long URL
- unsupported protocol
- malformed URL

Download behavior

Check:

- download starts correctly
- download completes
- incorrect URL
- unavailable resource
- timeout
- large file
- empty response
- incorrect content type
- corrupted response
- duplicate download

- repeated request
- concurrent download
- cancellation if supported
- retry behavior if supported

File validation

Check:

- filename
- extension
- MIME type
- file size
- downloaded content
- response headers
- Content-Disposition
- duplicate filenames
- special characters in filenames

Only test behaviors actually supported by the application.

16. API CHAINING TESTING

Identify dependent API flows.

For example:

```
Login
↓
Get Token
↓
Create Download
↓
Get Download ID
↓
Check Status
↓
Download File
↓
Delete / Cleanup
```

Create complete end-to-end API scenarios.

For every chain explain:

- What response value is extracted?
- Where is it used?
- Why is chaining required?
- What happens if one API fails?

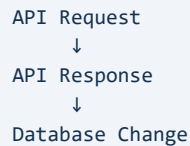
Create:

[API_TESTING/07_API_WORKFLOWS.md](#)

17. DATABASE VALIDATION

If the application uses a database, identify what API operations change.

For important APIs validate:



```
graph TD; A[API Request] --> B[API Response]; B --> C[Database Change];
```

Check:

- record creation
- record update
- record deletion
- status changes
- timestamps
- IDs
- relationships
- duplicate records

Do not modify production data.

Use only the local development/test database.

Create:

[API_TESTING/08_DATABASE_VALIDATION.md](#)

18. ERROR HANDLING TESTING

For every endpoint identify how errors are handled.

Test:

- validation error
- authentication error
- authorization error
- not found
- conflict
- timeout
- external service failure
- database failure
- malformed request

- unexpected server error

Check whether errors contain:

- correct status code
- useful message
- error code
- field-level validation
- consistent structure

Also check that sensitive information is NOT exposed.

Do not expose:

- passwords
- tokens
- secrets
- stack traces
- database credentials
- internal filesystem paths

Create:

[API_TESTING/09_ERROR_HANDLING.md](#)

19. SECURITY-ORIENTED API TESTING

Perform safe security validation against the local application.

Check for:

- authentication bypass
- authorization bypass
- IDOR-style access issues
- sensitive information exposure
- excessive data exposure
- missing input validation
- insecure headers
- weak error handling
- unrestricted input
- unsafe file handling
- malicious-looking input handling

For injection-style tests, use harmless test strings only.

Do not perform destructive exploitation.

Create:

[API_TESTING/10_API_SECURITY_TESTING.md](#)

20. PERFORMANCE TESTING

Perform basic API response-time validation.

Record:

```
Endpoint
Method
Request
Response Time
Status
Payload Size
Result
```

Categorize results:

```
Fast
Acceptable
Slow
Failed
Timeout
```

Do not claim performance benchmarks unless actually measured.

If load testing tools are available, explain how to gradually perform:

```
1 user
5 users
10 users
25 users
50 users
```

Only use realistic local/test environments.

Create:

[API_TESTING/11_API_PERFORMANCE_TESTING.md](#)

21. POSTMAN COLLECTION

Create a professional Postman collection representing the discovered APIs.

Organize folders:

```
URL Downloader API
├── Authentication
├── Health
├── Download
├── Download Status
├── Download Management
├── Error Scenarios
└── End-to-End Workflows
```

Use environment variables such as:

```
baseUr1
token
userId
downloadId
```

Do not hardcode secrets.

Add useful Postman tests for:

- status code
- response time
- Content-Type
- required response fields
- JSON schema where appropriate
- IDs
- response values used by later requests

Create:

[API_TESTING/postman/](#)

22. POSTMAN LEARNING MODE

For each Postman request, create documentation explaining:

Method

Why GET/POST/PUT/PATCH/DELETE is used.

URL

How the endpoint is constructed.

Headers

Why each header is required.

Body

Why each field exists.

Response

What the response means.

Tests

Why each assertion exists.

The objective is that I should be able to open Postman and manually reproduce the test myself.

23. MANUAL TEST EXECUTION DASHBOARD

Create a simple test execution tracker.

Example:

Test ID	Endpoint	Scenario	Expected	Actual	Status	Bug ID
-----	-----	-----	-----	-----	-----	-----
TC-001	POST /download	Valid URL	201	201	PASS	-
TC-002	POST /download	Empty URL	400	500	FAIL	BUG-001

Create:

`API_TESTING/test-execution/API_TEST_EXECUTION.md`

IMPORTANT:

Do not mark tests as PASS unless they have actually been executed.

Use:

```
NOT EXECUTED
PASS
FAIL
BLOCKED
```

24. BUG REPORT GENERATOR

Whenever a test fails, create a professional bug report.

Format:

```
Bug ID:
Title:
Severity:
Priority:
Environment:
Endpoint:
Method:

Precondition:

Steps to Reproduce:

Request:

Expected Result:

Actual Result:

HTTP Status:

Response:

Impact:

Evidence:

Possible Root Cause:

Developer Notes:
```

Create:

[API_TESTING/bugs/](#)

Use realistic QA terminology.

25. TEST COVERAGE MATRIX

Create a matrix showing coverage.

Example:

Endpoint	Positive	Negative	Boundary	Auth	Authorization	Response	Error	Security	DB	Performance
-----	-----	-----	-----	---	-----	-----	-----	--	-----	

Create:

[API_TESTING/12_API_COVERAGE_MATRIX.md](#)

Calculate actual coverage from the created test cases.

Do not fabricate execution coverage.

26. API TESTING REPORT

Create:

[API_TESTING/13_API_TEST_REPORT.md](#)

Include:

- Project overview
- Environment
- API count
- HTTP methods
- Test case count
- Executed count
- Passed count
- Failed count
- Blocked count
- Defects
- Critical defects
- Authentication results
- Authorization results
- Security observations
- Performance observations
- API coverage

- Recommendations
-

27. TEACH ME LIKE A MANUAL TESTER

Create:

[API_TESTING/LEARNING/API_TESTING_TRAINING.md](#)

Teach me from beginner to advanced.

Start with:

Level 1 — API Basics

Explain:

- What is an API?
- What is REST?
- Client vs server
- Endpoint
- Resource
- HTTP method
- Request
- Response
- Headers
- Body
- Query parameter
- Path parameter
- Authentication
- Authorization
- Status codes

Use examples from MY URL Downloader.

Level 2 — Manual API Testing

Teach:

- How to identify an endpoint
- How to create a request
- How to read a response
- How to validate status codes
- How to validate response body
- How to test negative scenarios
- How to find bugs

Level 3 — Postman

Teach:

- Collections
 - Environments
 - Variables
 - Authorization
 - Headers
 - Body
 - Tests
 - Pre-request scripts
 - Chaining
 - Collection Runner
-

Level 4 — Advanced API Testing

Teach:

- Contract testing
 - Schema validation
 - Authentication testing
 - Authorization testing
 - API security testing
 - Data validation
 - Integration testing
 - Performance testing
 - API mocking
 - Service virtualization
-

Level 5 — API Automation

After I understand manual testing, teach me how to automate the same tests.

Prefer the technologies already relevant to my QA learning path:

- Java
- Rest Assured
- TestNG
- Maven
- Postman
- Git

- GitHub Actions

Use the simplest architecture appropriate for this project.

Do NOT jump directly into complex frameworks.

28. REST ASSURED AUTOMATION

Create a separate folder:

`API_TESTING/automation/`

Build automation only AFTER understanding the API manually.

Structure appropriately, for example:

```
src/test/java
  tests
  clients
  models
  utilities
  assertions
  config
```

Use:

- Rest Assured
- TestNG
- Maven

Include:

- GET
- POST
- PUT/PATCH
- DELETE
- authentication
- request specification
- response validation
- JSON validation
- reusable methods
- test data
- logging
- reporting

Do not over-engineer.

Explain every class and method.

29. API AUTOMATION VS MANUAL TESTING

Create:

`API_TESTING/LEARNING/MANUAL_TO_AUTOMATION.md`

For each test show:

```
Manual Test
↓
Postman Test
↓
Rest Assured Test
↓
CI/CD Test
```

Example:

```
Manual:
Send POST request → verify 201

Postman:
pm.response.to.have.status(201)

Rest Assured:
given()
.when()
.post()
.then()
.statusCode(201);
```

Explain exactly what changed and what did not.

30. CI/CD

If GitHub Actions/Jenkins configuration is practical for this project, create a CI pipeline that:

1. checks out code
2. starts required services
3. starts application
4. waits for API availability
5. executes API tests
6. generates report
7. stores test artifacts
8. fails pipeline when critical tests fail

Create appropriate documentation.

Do not add CI/CD until the API tests work locally.

31. API TESTING FOLDER STRUCTURE

Create a clean structure similar to:

```
API_TESTING/
|
|— 00_PROJECT_ANALYSIS.md
|— 01_API_INVENTORY.md
|— 02_API_MAP.md
|— 03_MANUAL_API_TESTING_GUIDE.md
|— 04_AUTHENTICATION_TESTING.md
|— 05_AUTHORIZATION_TESTING.md
|— 06_STATUS_CODE_MATRIX.md
|— 07_API_WORKFLOWS.md
|— 08_DATABASE_VALIDATION.md
|— 09_ERROR_HANDLING.md
|— 10_API_SECURITY_TESTING.md
|— 11_API_PERFORMANCE_TESTING.md
|— 12_API_COVERAGE_MATRIX.md
|— 13_API_TEST_REPORT.md
|
|— test-cases/
|   |— positive-tests.md
|   |— negative-tests.md
|   |— boundary-tests.md
|
|— bugs/
|
|— test-execution/
|   |— API_TEST_EXECUTION.md
|
|— learning/
|   |— API_TESTING_TRAINING.md
|   |— MANUAL_TO_AUTOMATION.md
|
|— postman/
|   |— collection.json
|   |— environment.json
|
|— automation/
|   |— [Rest Assured project]
```

Adapt this structure if the existing repository already has a better organization.

Do not unnecessarily duplicate existing project structure.

32. IMPORTANT RULE — DO NOT MODIFY BUSINESS LOGIC

Unless explicitly required:

DO NOT change the application's business logic.

DO NOT rewrite the application.

DO NOT "fix" bugs automatically.

If you discover a defect:

```
Document it
↓
Reproduce it
↓
Create bug report
↓
```

Explain likely cause

Do not silently change the application to make the test pass.

33. IMPORTANT RULE — NEVER INVENT RESULTS

This is critical.

Never say:

PASS

unless the test has actually been executed and passed.

Never say:

API returns 200

unless verified from the application.

Never invent:

- endpoints
- request fields
- status codes
- database records
- performance results
- bugs
- test results

Use:

UNKNOWN
NOT VERIFIED
NOT EXECUTED

when evidence is unavailable.

34. TEST EXECUTION MODE

After generating the test cases, create a way for me to execute them manually.

For every test provide:

STEP 1
Open Postman.

STEP 2
Select method: POST

STEP 3
Enter URL:
{{baseUrl}}/...

```
STEP 4
Add headers.

STEP 5
Add body.

STEP 6
Click Send.

STEP 7
Check status code.

STEP 8
Check response.

STEP 9
Compare actual vs expected.

STEP 10
Mark:
PASS / FAIL / BLOCKED
```

Make these instructions beginner-friendly.

35. API TESTING CHECKLIST

Create:

`API_TESTING/API_TESTING_CHECKLIST.md`

Include:

Functional

- ☐ Happy path
- ☐ Negative scenarios
- ☐ Boundary values
- ☐ Validation
- ☐ CRUD
- ☐ Business rules

HTTP

- ☐ Method
- ☐ Status code
- ☐ Headers
- ☐ Content-Type
- ☐ Cache behavior

Authentication

- ☐ Valid token

- ☐ Missing token
- ☐ Invalid token
- ☐ Expired token

Authorization

- ☐ User permissions
- ☐ Resource ownership
- ☐ Admin access

Data

- ☐ Request data
- ☐ Response data
- ☐ Database data
- ☐ Data consistency

Security

- ☐ Sensitive data
- ☐ Input validation
- ☐ Access control
- ☐ Error leakage

Performance

- ☐ Response time
- ☐ Timeout
- ☐ Large payload
- ☐ Concurrent requests

Automation

- ☐ Postman
- ☐ Rest Assured
- ☐ TestNG
- ☐ Reports
- ☐ CI/CD

36. INTERVIEW PREPARATION

Create:

[API_TESTING/LEARNING/API_TESTING_INTERVIEW_QA.md](#)

Generate interview questions based specifically on this project.

Include:

Beginner

- What is API testing?
- What is REST?
- Difference between GET and POST?
- What are headers?
- What is a status code?
- What is JSON?

Intermediate

- How do you test a POST API?
- How do you test negative scenarios?
- How do you validate JSON?
- How do you test authentication?
- How do you test authorization?
- What is API chaining?
- What is schema validation?

Advanced

- How would you test a URL downloader API?
- How would you test download failures?
- How would you test timeout?
- How would you test large files?
- How would you test duplicate downloads?
- How would you validate API response against database?
- How do you automate API tests using Rest Assured?
- How do you integrate API automation with Jenkins/GitHub Actions?

For each question provide:

```
Simple interview answer
Real project example
Technical explanation
Possible follow-up question
```

37. DAILY LEARNING MODE

Create:

[API_TESTING/LEARNING/30_DAY_API_TESTING_PLAN.md](#)

Make it practical.

Days 1–5

API fundamentals

Days 6–10

Manual Postman testing

Days 11–15

Negative/boundary/security testing

Days 16–20

Advanced API testing

Days 21–25

Rest Assured

Days 26–28

Framework/reporting

Days 29–30

CI/CD + interview preparation

Every day must include:

```
Concept
URL Downloader exercise
Manual task
Expected learning
Interview question
```

38. FINAL OUTPUT

When analysis is complete, show me a summary:

```
PROJECT:
URL Downloader

TOTAL ENDPOINTS:
X

GET:
X

POST:
X

PUT:
X

PATCH:
X

DELETE:
X
```

AUTHENTICATED ENDPOINTS:

X

PUBLIC ENDPOINTS:

X

POSITIVE TEST CASES:

X

NEGATIVE TEST CASES:

X

BOUNDARY TEST CASES:

X

SECURITY TEST CASES:

X

API WORKFLOWS:

X

POSTMAN COLLECTION:

Created / Not Created

AUTOMATION:

Created / Not Created

CI/CD:

Created / Not Created

Then show:

WHAT I SHOULD DO FIRST

Give me exactly the first 5 manual API tests I should execute myself.

For each:

Test ID
Endpoint
Purpose
Postman steps
Request
Expected result
What I should learn

Do NOT overwhelm me with automation initially.

The learning order should be:

UNDERSTAND
↓
MANUALLY TEST
↓
FIND BUGS
↓
DOCUMENT
↓
POSTMAN
↓
AUTOMATE
↓
CI/CD

39. FINAL TEACHING PRINCIPLE

Treat me as a manual QA engineer learning API testing, not as an experienced API automation engineer.

Whenever you introduce a technical concept:

1. Explain it simply.
2. Show it using my URL Downloader.
3. Show how I can test it manually.
4. Explain expected vs actual.
5. Show how to identify a defect.
6. Only then show automation.

The final objective is:

"After completing this project, I should be able to independently discover APIs, understand requests/responses, design complete manual API test cases, execute them in Postman, identify and report defects, validate database behavior, perform API security/negative testing, and finally automate the same tests using Rest Assured + TestNG and run them through CI/CD."

Start by inspecting the existing URL Downloader project.

Do not modify application business logic.

Do not invent API behavior.

Do not mark unexecuted tests as passed.

Begin with:

PHASE 1 — PROJECT ANALYSIS + COMPLETE API ENDPOINT DISCOVERY.